

Döntési Napló és rövid reflexió

Döntési napló

#	Döntési pont	Amit választottam	Miért	Milyen alternatívát vettem el
1	Backend technológia kiválasztása	C# és ASP.NET Core	C# nyelven már rendelkeztem programozási tapasztalattal, ezért ezt a technológiát tudtam a leggyorsabban használni a feladat megvalósításához.	Más backend technológiák, például Node.js vagy Java használata.
2	Adatbázis kiválasztása	SQLite	A feladat adatbázis-használatot követelt meg, a SQLite pedig egyszerűen használható .NET és Entity Framework Core mellett, és nem igényel külön adatbázis-szervert.	Külső adatbázis-szerver, például PostgreSQL vagy MySQL használata.
3	Interfész kiválasztása	REST jellegű HTTP API	A feladat API-alapú megoldásként egyszerűen tesztelhető, és az egyes műveletek jól elkülöníthetők HTTP metódusokkal.	Más felhasználói interfész.

4	Foglalási ütközések kezelése	A foglalás létrehozásakor ellenőrzöm az adott parkolóhely meglévő foglalásait, és átfedés esetén elutasítom az új foglalást.	Így ugyanahhoz a parkolóhelyhez nem hozható létre időben átfedő foglalás.	Az ütközések ellenőrzésének elhagyása vagy későbbi kezelése
5	Érvényes foglalási időpontok meghatározása	A foglalás létrehozásakor ellenőrzöm, hogy a kezdeti idő nagyobb-e, mint az aktuális idő, illetve, hogy a kezdeti idő kisebb-e, mint a végidő.	Így csak érvényes idejű foglalás hozható létre.	Múlt idejű, vagy ellentmondásos foglalások elfogadása
6	Tesztadatbázis megoldása	A unit tesztekhez memóriában futó SQLite adatbázist használok.	Így a tesztek elkülönülnek az alkalmazás normál adatbázisától, és minden tesztelési környezetben létrehozható a szükséges adatbázis.	A fejlesztés során használt normál SQLite adatbázis közvetlen használata a tesztekhez
7	Kezdeti parkolóhely létrehozása	Az alkalmazás indulásakor ellenőrzöm, hogy van-e parkolóhely az adatbázisban, és ha nincs, létrehozok egy alapértelmezett parkolóhelyet.	A feladat előírja, hogy a rendszer inicializált, feltöltött állapotban induljon.	Üres adatbázissal történő indulás

Rövid reflexió

A feladat során az egyik legnagyobb kihívást az jelentette, hogy korábban nem készítettem backend API-t és unit teszteket, ezért ezeket a technológiákat a feladat elkészítése közben kellett megismernem. A fejlesztés során különösen az adatbázis és az Entity Framework Core használatának, valamint a teszteléshez szükséges külön SQLite adatbázis létrehozásának megértése jelentett új feladatot. A megoldás elkészítése közben több esetet is teszteltem, például érvénytelen időintervallummal, nem létező parkolóhellyel és érvényes adatokkal létrehozott foglalásokat. A unit tesztek segítségével ellenőriztem, hogy a foglalási logika a várt módon működik. A legnagyobb kihívás számomra a unit tesztelés kialakítása volt, mivel korábban nem dolgoztam xUnit tesztekkel.

AI-eszköz használata

A fejlesztés során ChatGPT-t használtam tanulási és problémamegoldási segítségként. Segítséget kértem többek között az ASP.NET Core API működésének, az Entity Framework Core használatának, a unit tesztelésnek és a memóriában futó SQLite tesztadatbázis kialakításának megértéséhez. A feladatokat elsősorban magamtól próbáltam megoldani, az AI-t főként a gondolatmenet és az új technológiák megértéséhez, illetve szükség esetén a felmerülő technikai problémák tisztázásához használtam. Az AI segítségével kapott javaslatokat nem automatikusan vettem át, hanem a működésüket teszteltem és a saját megoldásomban alkalmaztam. A fejlesztés során a kódot saját környezetben futtattam, és a működését manuális API-kérésekkel, valamint unit tesztekkel ellenőriztem.

Az AI-asszisztenssel folytatott beszélgetés nyers prompt history-ja külön mellékletként kerül beadásra.